

Pandas LEVEL 16 — Combining DataFrames

Complete line-by-line guide with examples and outputs

Combining DataFrames is essential in real-world data analysis because data is often stored in multiple tables or files. Pandas provides `concat()`, `merge()`, `join()`, and `combine_first()` for bringing data together.

1. Why Combine DataFrames?

Code:

```
import pandas as pd

df1 = pd.DataFrame({
    "Name": ["Amit", "Riya"],
    "Salary": [30000, 40000]
})

df2 = pd.DataFrame({
    "Name": ["John", "Neha"],
    "Salary": [35000, 45000]
})

print(df1)
print(df2)
```

Output:

```
df1
   Name  Salary
0  Amit   30000
1  Riya   40000

df2
   Name  Salary
0  John   35000
1  Neha   45000
```

We can combine these two DataFrames into one DataFrame.

2. concat() — Basic Idea

Code:

```
df1 = pd.DataFrame({"Name": ["Amit", "Riya"], "Age": [22, 23]})
df2 = pd.DataFrame({"Name": ["John", "Neha"], "Age": [24, 25]})

result = pd.concat([df1, df2])

print(result)
```

Output:

```
   Name  Age
0  Amit   22
1  Riya   23
0  John   24
1  Neha   25
```

`concat()` combines DataFrames. By default, it keeps the original indexes.

3. Vertical Concatenation

Code:

```
result = pd.concat([df1, df2], axis=0)

print(result)
```

Output:

	Name	Age
0	Amit	22
1	Riya	23
0	John	24
1	Neha	25

axis=0 means rows are added vertically.

4. Vertical Concatenation with ignore_index=True

Code:

```
result = pd.concat([df1, df2], axis=0, ignore_index=True)

print(result)
```

Output:

	Name	Age
0	Amit	22
1	Riya	23
2	John	24
3	Neha	25

ignore_index=True creates a new continuous index.

5. Horizontal Concatenation

Code:

```
df1 = pd.DataFrame({"Name": ["Amit", "Riya"]})
df2 = pd.DataFrame({"Age": [22, 23]})

result = pd.concat([df1, df2], axis=1)

print(result)
```

Output:

	Name	Age	
0	Amit	22	0
1	Riya	23	1

axis=1 combines columns side by side using the index.

6. merge() — Basic Idea

Code:

```
employees = pd.DataFrame({
    "EmpID": [1, 2, 3],
    "Name": ["Amit", "Riya", "John"]
})

salary = pd.DataFrame({
    "EmpID": [1, 2, 3],
    "Salary": [30000, 40000, 50000]
})
```

```
result = pd.merge(employees, salary, on="EmpID")

print(result)
```

Output:

	EmpID	Name	Salary
0	1	Amit	30000
1	2	Riya	40000
2	3	John	50000

merge() combines related tables using a common key.

7. Inner Join

Code:

```
left = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Amit", "Riya", "John"]
})

right = pd.DataFrame({
    "ID": [2, 3, 4],
    "Salary": [40000, 50000, 60000]
})

result = pd.merge(left, right, on="ID", how="inner")

print(result)
```

Output:

	ID	Name	Salary
0	2	Riya	40000
1	3	John	50000

Inner join keeps only matching IDs from both DataFrames.

8. Left Join

Code:

```
result = pd.merge(left, right, on="ID", how="left")

print(result)
```

Output:

	ID	Name	Salary
0	1	Amit	NaN
1	2	Riya	40000.0
2	3	John	50000.0

Left join keeps every row from the left DataFrame.

9. Right Join

Code:

```
result = pd.merge(left, right, on="ID", how="right")

print(result)
```

Output:

	ID	Name	Salary
--	----	------	--------

0	2	Riya	40000
1	3	John	50000
2	4	NaN	60000

Right join keeps every row from the right DataFrame.

10. Outer Join

Code:

```
result = pd.merge(left, right, on="ID", how="outer")

print(result)
```

Output:

	ID	Name	Salary
0	1	Amit	NaN
1	2	Riya	40000.0
2	3	John	50000.0
3	4	NaN	60000.0

Outer join keeps all IDs from both DataFrames.

11. Cross Join

Code:

```
colors = pd.DataFrame({"Color": ["Red", "Blue"]})
sizes = pd.DataFrame({"Size": ["S", "M", "L"]})

result = pd.merge(colors, sizes, how="cross")

print(result)
```

Output:

	Color	Size
0	Red	S
1	Red	M
2	Red	L
3	Blue	S
4	Blue	M
5	Blue	L

Cross join creates every possible combination of rows.

12. Merge on One Column

Code:

```
df1 = pd.DataFrame({
    "EmployeeID": [101, 102, 103],
    "Name": ["Amit", "Riya", "John"]
})

df2 = pd.DataFrame({
    "EmployeeID": [101, 102, 103],
    "Department": ["IT", "HR", "Finance"]
})

result = pd.merge(df1, df2, on="EmployeeID")

print(result)
```

Output:

	EmployeeID	Name	Department
0	101	Amit	IT
1	102	Riya	HR
2	103	John	Finance

on="EmployeeID" tells Pandas which common column to use.

13. Merge on Multiple Columns

Code:

```
df1 = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "City": ["Pune", "Mumbai", "Delhi"],
    "Score": [90, 85, 88]
})

df2 = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "City": ["Pune", "Mumbai", "Delhi"],
    "Salary": [50000, 45000, 60000]
})

result = pd.merge(df1, df2, on=["Name", "City"])

print(result)
```

Output:

	Name	City	Score	Salary
0	Amit	Pune	90	50000
1	Riya	Mumbai	85	45000
2	John	Delhi	88	60000

Use a list inside on= when more than one column forms the matching key.

14. Merge Using Index

Code:

```
df1 = pd.DataFrame(
    {"Name": ["Amit", "Riya", "John"]},
    index=[101, 102, 103]
)

df2 = pd.DataFrame(
    {"Salary": [30000, 40000, 50000]},
    index=[101, 102, 103]
)

result = pd.merge(
    df1, df2,
    left_index=True,
    right_index=True
)

print(result)
```

Output:

	Name	Salary
101	Amit	30000
102	Riya	40000
103	John	50000

left_index=True and right_index=True tell Pandas to match the indexes.

15. join() — Basic Example

Code:

```
df1 = pd.DataFrame(
    {"Name": ["Amit", "Riya", "John"]},
    index=[101, 102, 103]
)

df2 = pd.DataFrame(
    {"Salary": [30000, 40000, 50000]},
    index=[101, 102, 103]
)

result = df1.join(df2)

print(result)
```

Output:

	Name	Salary
101	Amit	30000
102	Riya	40000
103	John	50000

join() is convenient when DataFrames should be combined using their indexes.

16. join() with how='inner'

Code:

```
df1 = pd.DataFrame({"Name": ["Amit", "Riya", "John"]}, index=[1, 2, 3])
df2 = pd.DataFrame({"Salary": [40000, 50000]}, index=[2, 3])

result = df1.join(df2, how="inner")

print(result)
```

Output:

	Name	Salary
2	Riya	40000
3	John	50000

The inner join keeps only indexes present in both DataFrames.

17. combine_first()

Code:

```
df1 = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, None, 25]
})

df2 = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 23, 25]
})

result = df1.combine_first(df2)

print(result)
```

Output:

	Name	Age
--	------	-----

```
0  Amit  22.0
1  Riya  23.0
2  John  25.0
```

`combine_first()` fills missing values in the first DataFrame using values from the second DataFrame.

18. concat() with Different Columns

Code:

```
df1 = pd.DataFrame({
    "Name": ["Amit", "Riya"],
    "Age": [22, 23]
})

df2 = pd.DataFrame({
    "Name": ["John", "Neha"],
    "Salary": [50000, 60000]
})

result = pd.concat([df1, df2], ignore_index=True)

print(result)
```

Output:

	Name	Age	Salary
0	Amit	22.0	NaN
1	Riya	23.0	NaN
2	John	NaN	50000.0
3	Neha	NaN	60000.0

`concat()` aligns columns by column name and fills missing columns with NaN.

19. concat() with join='inner'

Code:

```
df1 = pd.DataFrame({
    "Name": ["Amit", "Riya"],
    "Age": [22, 23]
})

df2 = pd.DataFrame({
    "Name": ["John", "Neha"],
    "Salary": [50000, 60000]
})

result = pd.concat([df1, df2], join="inner", ignore_index=True)

print(result)
```

Output:

	Name
0	Amit
1	Riya
2	John
3	Neha

`join="inner"` keeps only columns common to all DataFrames.

20. Important Difference: concat() vs merge() vs join()

Code:

```
# concat() -> combines DataFrames along rows or columns
# merge() -> combines related tables using one or more keys
# join() -> commonly combines DataFrames using indexes
# combine_first() -> fills missing values using another DataFrame
```

Output:

```
Quick rule:
concat = stack / place side by side
merge = SQL-style key-based combination
join = convenient index-based combination
combine_first = fill missing values from another DataFrame
```

21. Interview Example — Employee + Department

Code:

```
employees = pd.DataFrame({
    "EmpID": [1, 2, 3, 4],
    "Name": ["Amit", "Riya", "John", "Neha"],
    "DeptID": [10, 20, 10, 30]
})

departments = pd.DataFrame({
    "DeptID": [10, 20, 30],
    "Department": ["IT", "HR", "Finance"]
})

result = pd.merge(
    employees,
    departments,
    on="DeptID",
    how="left"
)

print(result)
```

Output:

	EmpID	Name	DeptID	Department
0	1	Amit	10	IT
1	2	Riya	20	HR
2	3	John	10	IT
3	4	Neha	30	Finance

This is a common real-world use of `merge()`: adding lookup information to a main dataset.

22. Quick Reference

```
pd.concat([df1, df2])                # vertical by default
pd.concat([df1, df2], axis=0)         # vertical
pd.concat([df1, df2], axis=1)         # horizontal
pd.concat([df1, df2], ignore_index=True) # reset index

pd.merge(df1, df2, on="ID")           # inner join by default
pd.merge(df1, df2, on="ID", how="inner")
pd.merge(df1, df2, on="ID", how="left")
pd.merge(df1, df2, on="ID", how="right")
pd.merge(df1, df2, on="ID", how="outer")
pd.merge(df1, df2, how="cross")
pd.merge(df1, df2, on=["A", "B"])     # multiple keys
pd.merge(df1, df2, left_index=True, right_index=True)

df1.join(df2)                         # index-based join
df1.join(df2, how="inner")

df1.combine_first(df2)                # fill missing values
```

23. Join Types — Easy Memory Trick

Inner: only matching rows.

Left: everything from left + matches from right.

Right: everything from right + matches from left.

Outer: everything from both sides.

Cross: every possible pair of rows.

24. Key Interview Points

1. `concat()` is useful for stacking datasets or combining columns.
2. `merge()` is best for SQL-style key-based relationships.
3. `join()` is especially convenient for index-based combinations.
4. `combine_first()` is useful for filling missing values from another DataFrame.
5. Always check duplicate keys before merging because duplicate keys can create multiple rows.
6. Use `validate=` in `merge()` when you want Pandas to check expected relationship types.